

编译原理大作业 1：词法和语法分析工具

——类 Rust 语言的词法与语法分析器设计与实现

2351078 李堂辉

2351837 高胜寒

2352197 冉子易

(按学号排序)

同济大学

2026 年 4 月 14 日

目录

项目概述

词法分析

语法分析

系统架构

测试与演示

思考与总结

环境与执行

任务要求

大作业目标

设计并实现面向类 **Rust** 语言的词法分析和语法分析工具

基本功能

- 词法分析：识别 Token
- 语法分析：构建 AST
- 输出分析结果

覆盖规则

- 必需：0.1–5.1（18 条）
- 扩展：5.2–9.3（16 条）
- 合计实现 **34** 条产生式规则

技术选型

实现语言

Rust

- 类型安全，内存安全
- 枚举类型天然适合 Token 和 AST
- 模式匹配简化分析逻辑

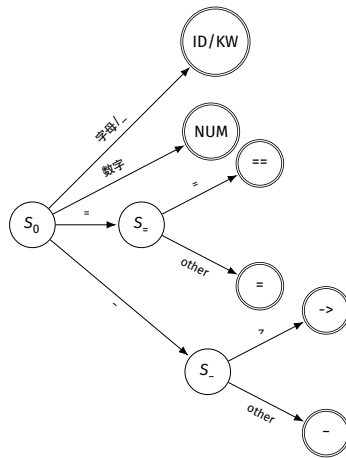
分析方法

- 词法：手写 Cursor 扫描器
- 语法：递归下降分析器
- 无外部依赖（纯标准库）

词法单元分类

类别	内容
关键字	i32, let, if, else, while, return, mut, fn, for, in, loop, break, continue
标识符	(字母 _)(字母 数字 _)*, 不与关键字相同
数值	数字(数字)*
赋值号	=
算符	+ - * / == > >= < <= != &
界符	() { } []
分隔符	; : ,
特殊符号	-> . ..
注释	// ... 和 /* ... */
结束符	#

词法分析器——状态转换



核心策略：读取字符 → 根据首字符分派 → 必要时前瞻一个字符区分

关键字 vs 标识符

识别策略

先按标识符规则扫描完整单词，再查关键字表

示例 1: if123

- 首字符 i, 进入标识符状态
- 继续读入 f, 1, 2, 3
- 得到 if123
- 不在关键字表中 ⇒ 标识符

示例 2: if=123

- 读入 i, f, 下一个是 = (非字母数字)
- 得到 if ⇒ 关键字
- 然后 = ⇒ 赋值号
- 然后 123 ⇒ 整数

语法分析方法：递归下降

方法简介

递归下降分析（Recursive Descent Parsing）是一种自顶向下的分析方法。为文法中每个非终结符编写一个解析函数，通过前瞻 Token 选择产生式。

优点

- 代码直观、易于调试
- 可利用上下文消歧
- 错误信息精确

处理步骤

- ① 消除左递归
- ② 确定各产生式的 FIRST 集
- ③ 编写递归函数

左递归消除

原始文法（以加减表达式为例）：

〈加减表达式〉 → 〈加减表达式〉 〈加减运算符〉 〈项〉 | 〈项〉

消除后（改写为迭代循环）：

```
1 fn parse_add_sub(&mut self) -> Result<Expr> {  
2     let mut left = self.parse_mul_div()?;  
3     loop {  
4         let op = match self.peek() {  
5             Plus => BinOp::Add,  
6             Minus => BinOp::Sub,  
7             _ => break,  
8         };  
9         self.advance();  
10        let right = self.parse_mul_div()?;  
11        left = Expr::BinaryOp(left, op, right);  
12    }  
13    Ok(left)  
14 }
```

运算符优先级

通过函数调用层级实现优先级：

层级	运算符	函数
1 (低)	..	parse_range_expr
2	== != < <= > >=	parse_comparison
3	+ -	parse_add_sub
4	* /	parse_mul_div
5	* & &mut (一元)	parse_unary
6	[] .N (后缀)	parse_postfix
7 (高)	字面量/括号	parse_primary

低优先级函数调用高优先级函数 ⇒ 高优先级运算先绑定

增强型语义分析与安全性

核心目标

在语法分析过程中集成语义校验，提升工具的健壮性。

可变性校验

- `let mut` 严格声明
- 禁止对不可变变量赋值
- 支持解引用可变性校验
- 指向 `&mut T` 的解引用可写

类型与迭代校验

- **类型一致性**：赋值与声明时的类型必须完全匹配
- **迭代合法性**：`for` 循环仅支持 `Range` 或 `Array`

已实现的语法规则总览

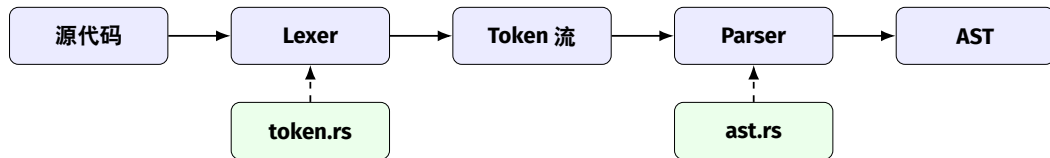
基础（必需）

- 0.1 变量属性 (mut/空)
- 0.2 类型 (i32)
- 0.3 左值 (标识符)
- 1.1 基础程序
- 1.2 语句/语句串
- 1.3 返回语句
- 1.4 函数输入（形参列表）
- 1.5 函数输出（返回类型）
- 2.0 变量声明
- 2.1 变量声明语句
- 2.2 赋值语句
- 3.1–3.5 表达式/运算/函数调用
- 4.1 if 选择
- 5.0–5.1 循环/while

扩展（额外实现）

- 2.3 变量声明赋值
- 4.2–4.3 else / else if
- 5.2 for 循环
- 5.3 loop 循环
- 5.4 break / continue
- 6.1 不可变变量
- 6.2–6.3 不可变/可变引用
- 6.4 借用（解引用）
- 7.0–7.4 函数表达式块
- 8.1–8.3 数组
- 9.1–9.3 元组

模块架构



- **token.rs**: Token 类型定义
- **lexer.rs**: 词法扫描器

- **ast.rs**: AST 节点 + 打印
- **parser.rs**: 递归下降分析器

测试用例：基础程序 (1.1-1.5)

```
1 fn program_1_1() {  
2 }  
3 fn program_1_2() {  
4     ;;;;;;  
5 }  
6 fn program_1_3() {  
7     return ;  
8 }  
9 fn program_1_4(mut a:i32) {  
10 }  
11 fn program_1_5() -> i32 {  
12     return 1;  
13 }
```

AST 输出：

```
FunctionDecl: program_1_1()  
  Block
```

```
FunctionDecl: program_1_2()  
  Block  
    EmptyStmt (x6)
```

```
FunctionDecl: program_1_3()  
  Block  
    Return (void)
```

```
FunctionDecl: program_1_4(mut a: i32)  
  Block
```

```
FunctionDecl: program_1_5() -> i32  
  Block  
    Return  
      Int(1)
```

测试用例：变量与赋值 (2.1-2.3)

```
1 fn program_2_1() {  
2   let mut a;  
3   let mut b:i32;  
4 }  
5 fn program_2_2(mut a:i32) {  
6   a=32;  
7 }  
8 fn program_2_3() {  
9   let mut a=1;  
10  let mut b:i32=1;  
11 }
```

AST 输出:

FunctionDecl: program_2_1()

Block

VarDecl: let mut a

VarDecl: let mut b: i32

FunctionDecl: program_2_2(mut a: i32)

Block

Assign

Ident(a)

Int(32)

FunctionDecl: program_2_3()

Block

VarDeclAssign: let mut a = ...

Int(1)

VarDeclAssign: let mut b: i32 = ...

Int(1)

测试用例：控制流 (4.1–4.3, 5.1)

```
1 fn program_4_3(a:i32) -> i32 {  
2     if a>0 {  
3         return a+1;  
4     } else if a<0 {  
5         return a-1;  
6     } else {  
7         return 0;  
8     }  
9 }  
10 fn program_5_1(mut n:i32) {  
11     while n>0 {  
12         n=n-1;  
13     }  
14 }
```

AST 输出:

FunctionDecl: program_4_3(a: i32)

Block

If

BinOp(>)

Ident(a), Int(0)

Block

Return BinOp(+) ...

ElseIf

If

BinOp(<)

Ident(a), Int(0)

...

Else

Block

Return Int(0)

FunctionDecl: program_5_1(mut n: i32)

Block

While

BinOp(>): Ident(n), Int(0)

Block

Assign: n = n-1

测试用例：数组与元组 (8.1-9.3)

```
1 fn program_8_2(mut a:[i32;3]) {  
2     a=[1,2,3];  
3 }  
4 fn program_8_3(mut a:[i32;3]) {  
5     let mut b:i32=a[0];  
6     a[0]=1;  
7 }  
8 fn program_9_2(mut a:(i32,i32)) {  
9     a=(1,2);  
10 }  
11 fn program_9_3(mut a:(i32,i32)) {  
12     let mut b:i32=a.0;  
13     a.0=1;  
14 }
```

AST 输出：

FunctionDecl: program_8_2(...)

Block

Assign

Ident(a)

ArrayLiteral(3 elems)

Int(1), Int(2), Int(3)

FunctionDecl: program_8_3(...)

Block

VarDeclAssign: let mut b: i32

Index: Ident(a), Int(0)

Assign

Index: Ident(a), Int(0)

Int(1)

... (元组类似)

测试结果

全部测试通过

28 个测试函数覆盖规则 0.1 – 9.3，全部成功通过词法分析和语法分析。

统计信息

- 词法分析：正确识别所有关键字、标识符、数值、运算符、界符等
- 语法分析：成功构建完整 AST，无报错
- 程序输出：“语法分析成功！”

思考：LR(1) 的局限

问题

PPT 提到“存在极个别的羁绊会使得 LR(1) 无法处理”

典型案例：表达式块 vs 语句块

- 遇到 { 时，LR(1) 无法确定是语句块还是表达式块
- 递归下降可以通过调用上下文区分
- LR(1) 中可尝试改写产生式或使用 GLR 来处理

元组与括号的歧义

- (expr) 是括号表达式
- (expr,) 是单元素元组
- 需要看到逗号后才能区分 ⇒ 需要更多前瞻

项目编译与执行

编译环境

- Rust 1.70+, Cargo
- LaTeX (XeLaTeX), latexmk (用于生成报告)

常用编译与执行命令

- `cargo build --release`: 编译生成 release 二进制文件
- `cargo run --release`: 运行内置语法树解析测试
- `cargo run --release -- tests/test_all.rs.txt`: 指定外部程序测试

总结

完成内容

- 使用 **Rust** 实现了类 Rust 语言的词法分析器和语法分析器
- 词法分析采用手写 **Cursor 扫描器**，支持 13 种关键字和所有词法单元
- 语法分析采用**递归下降**方法，消除左递归，正确处理运算符优先级
- 实现了从基础规则（0.1-5.1）到扩展规则（5.2-9.3）的**全部 34 条**产生式
- 所有 28 个测试用例**全部通过**

个人收获

- 深入理解了词法分析的状态机原理和语法分析的递归下降方法
- 掌握了左递归消除和运算符优先级的处理技巧
- 体会到 Rust 枚举 + 模式匹配在编译器前端实现中的优势

参考资料

- 陈火旺. 程序设计语言编译原理（第 3 版）. 国防工业出版社.
- Alfred V. Aho et al. Compilers: Principles, Techniques and Tools (Second Edition). 赵建华等译. 机械工业出版社.
- The Rust Reference & Rust By Example.

谢谢!